

UNIVERSIDAD ANDRÉS BELLO
FACULTAD DE INGENIERÍA
Departamento de Ingeniería Informática

ENTREGA 2:
DISEÑO E IMPLEMENTACIÓN DE UN BACKEND SEGURO COMPLETO

AUTOR
Sebastián Sepúlveda Ríos
19.026.688-5

PROFESOR
Ernesto Vivanco Tapia

NRC
8481

09 de Noviembre del 2026
Santiago - Chile

Tabla de Contenidos

1. Módulos. **4**

1.1. Módulo 1: Servicios Derivados del Frontend. 4
1.1.1. Matriz de Servicios. 4
1.1.2. Ejemplos de Contratos JSON. 4
1.2. Módulo 2: Clases y Cardinalidades del Dominio. 5
1.2.1. Diagrama UML de Clases. 5
1.2.2. Tabla de Relaciones y Multiplicidades. 5
1.2.3. Correspondencia entre Operación UML e ID de Servicio. 6
1.3. Módulo 3: Sistemas e Integraciones. 6
1.3.1. Mapa de Componentes. 6
1.3.2. Matriz de Responsabilidades y Dependencias. 7
1.3.3. Flujo de Integración (Secuencia). 7
1.4. Módulo 4: Stakeholders y Permisos HTTP. 7
1.4.1. Registro de Stakeholders. 7
1.4.2. Matriz de Permisos por Rol y Recurso. 8
1.5. Módulo 5: Clases del API Gateway. 8
1.5.1. Diagrama UML de Gateway y Políticas. 8
1.5.2. Fronteras de Confianza y Autorización. 9
1.5.3. Recorrido de Solicitudes. 9
1.6. Módulo 6: Rutas y Secretos de Integración. 10
1.6.1. Gestión de Secretos y Bóveda (Vault). 10
1.6.2. Protección de Rutas Internas. 10
1.7. Módulo 7: Protección de Datos y Privacidad. 10
1.7.1. Clasificación y Minimización de Datos. 10
1.7.2. Derechos del Titular y Cumplimiento Normativo (Compliance). 11
1.8. Módulo 8: Separación privada y agregaciones GraphQL. 11
1.8.1. Mapa de Clasificación de Servicios. 11
1.8.2. Esquema, Queries y Resolvers GraphQL. 11
1.8.3. Matriz de Autorización por Recurso y Campo. 13
1.8.4. Evidencia e Integración desde el Frontend. 13
1.9. Módulo 9: Analítica y Comunicación gRPC (ADR de Aplicabilidad). 14
1.9.1. Registro de Decisión de Arquitectura (ADR-001). 14
1.9.2. Caracterización de Carga y Medición de Rendimiento. 14
1.10. Módulo 10: Backend coCpleto e Integración Verificable. 14
1.10.1. Orquestación y Despliegue con Docker Compose. 15
1.10.2. Matriz de Pruebas de Integración y Verificación de Reglas. 15

Índice de Figuras

1. Diagrama de Clases del Dominio y Servicios Públicos de Happy Kids 5
2. Mapa de Microservicios y Enrutamiento del API Gateway 6
3. Diagrama de Secuencia: Colaboración Interna para la Creación de un Pedido 7
4. Diagrama de Clases del API Gateway, Políticas y Dependencias 9

Índice de Tablas

1. Matriz de Operaciones y Servicios del Frontend 4
2. Justificación detallada de Cardinalidades y Reglas de Negocio 6
3. Trazabilidad entre Diseño de Software y Servicios Expuestos al Frontend 6
4. Definición de Microservicios, Autoridad de Datos y Tolerancia a Fallos 7
5. Identificación de Actores y sus Roles Operativos 8
6. Control de Acceso Basado en Roles (RBAC) y Propiedad de Recursos 8
7. Matriz de Integración Segura y Gestión de Secretos 10
8. Mapa de Clasificación de Servicios, Responsabilidades y Límites de Datos 11
9. Matriz de Autorización por Rol, Recurso y Campos en GraphQL 13
10. Resultados de Pruebas de Carga en el Backend de Happy Kids 14
11. Matriz de Pruebas de Integración, Seguridad y Reglas de Negocio 16

1. Módulos.

1.1. Módulo 1: Servicios Derivados del Frontend.

En este módulo se busca identificar qué operación necesita cada pantalla del e-commerce *Happy Kids* y qué datos atraviesan la frontera entre el navegador y el backend.

1.1.1. Matriz de Servicios.

ID	Pantalla y Acción	Contrato (Verbo y Ruta)	Entrada / Salida	Reglas / Acceso
S01	**P01 Catálogo** → Cargar productos disponibles	**GET** /api/v1/products	**In:** Sin cuerpo. **Out:** 200 OK con lista de {id, nombre, precio, emoji, desc}.	**Regla:** Retorna solo juguetes con stock > 0. **Acceso:** Anónimo. Datos Públicos.
S02	**P02 Carrito** → Confirmar pedido	**POST** /api/v1/orders	**In:** JSON con lista de {producto_id, cantidad}. **Out:** 201 Created con {order_id, total, estado}.	**Regla:** Restar stock. Falla si no hay stock (400). **Acceso:** Autenticado. Datos Transaccionales.
S03	**P03 Registro** → Crear cuenta nueva	**POST** /api/v1/clients	**In:** JSON con {nombre, email, password, direccion}. **Out:** 201 Created con {client_id, email}.	**Regla:** El email debe ser único. Cifrar password (409). **Acceso:** Anónimo. Datos Personales (Sensibles).
S04	**P04 Checkout** → Iniciar pago	**POST** /api/v1/payments	**In:** JSON con {order_id, metodo_pago}. **Out:** 200 OK con {payment_id, status}.	**Regla:** Verificar que la orden exista y pertenezca al usuario (403/404). **Acceso:** Autenticado. Datos Sensibles.

Tabla 1: Matriz de Operaciones y Servicios del Frontend

Los valores numéricos en la columna de Entrada/Salida y Reglas corresponden a los códigos de estado HTTP estándar (ej. **200 OK** para éxito, **201 Created** para recurso generado exitosamente, **400 Bad Request** o **409 Conflict** para errores de validación de negocio).

1.1.2. Ejemplos de Contratos JSON.

Contrato S03 (Crear Cliente - POST /api/v1/clients)

Entrada (Enviado por el navegador):

```
{
  "nombre": "Ana",
  "email": "ana@ejemplo.com",
  "password": "PasswordSegura123!",
  "direccion": "Las Condes 123"
}
```

Salida (Respuesta del Backend - 201 Created):

```
{
  "mensaje": "Cuenta creada exitosamente",
  "perfil": {
    "client_id": "CLI-002",
    "nombre": "Ana",
    "email": "ana@ejemplo.com",
    "direccion": "Las Condes 123"
  }
}
```

1.2. Módulo 2: Clases y Cardinalidades del Dominio.

En este módulo el objetivo es representar las entidades necesarias para las operaciones del frontend de *Happy Kids* y los servicios que las exponen, definiendo relaciones coherentes con las reglas de negocio del caso.

1.2.1. Diagrama UML de Clases.

El siguiente diagrama ilustra las entidades del dominio, los servicios públicos (con el estereotipo ≪service≫) y las dependencias entre ellos. La separación de responsabilidades garantiza un bajo acoplamiento entre la gestión de usuarios, el inventario y las transacciones.

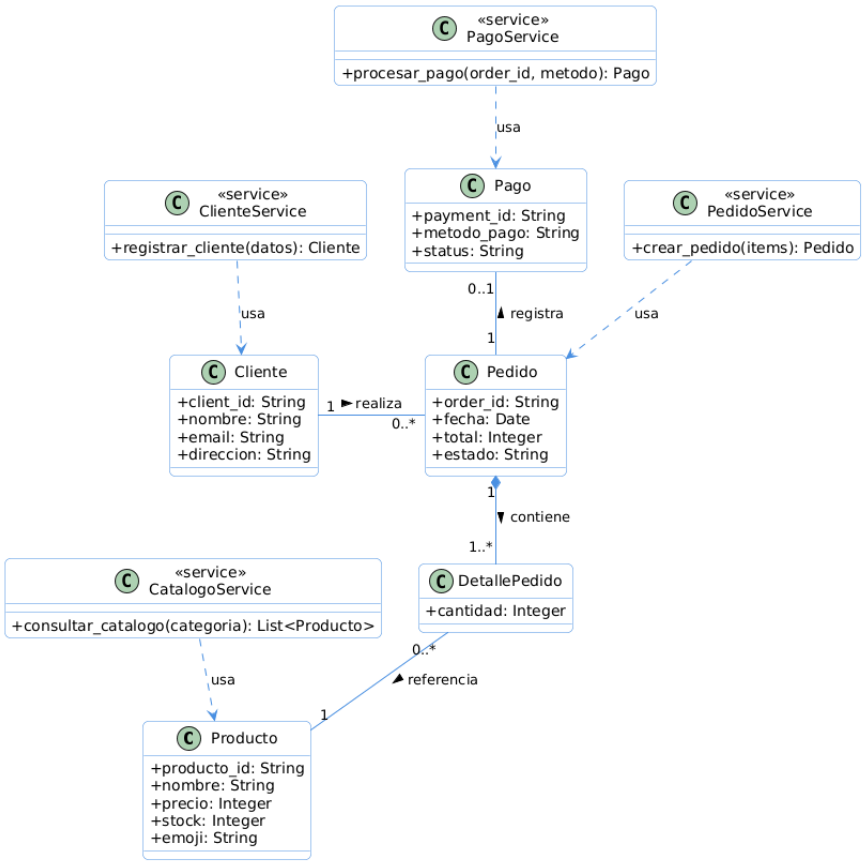


Figura 1: Diagrama de Clases del Dominio y Servicios Públicos de Happy Kids

1.2.2. Tabla de Relaciones y Multiplicidades.

Las cardinalidades del diagrama no son arbitrarias; responden a reglas estrictas del modelo de negocio del e-commerce *Happy Kids*, garantizando la integridad referencial de los datos:

Clase Origen	Clase Destino	Multiplicidad	Justificación de la regla de negocio
Cliente	Pedido	1 a 0..*	Un cliente registrado en la plataforma puede estar solo vitrineando sin haber comprado nada aún (0), o tener un historial con múltiples compras a lo largo del tiempo (*). Por trazabilidad y auditoría, un pedido siempre debe pertenecer a un único (1) cliente.
Pedido	DetallePedido	1 a 1..*	**(Composición fuerte)** Un pedido confirmado en el checkout no tiene sentido comercial si está vacío; debe contener al menos un juguete (1..*). Si un pedido se anula o elimina del sistema, sus detalles (líneas de compra) desaparecen estructuralmente con él.
DetallePedido	Producto	0..* a 1	Cada línea del carrito de compras (detalle) debe apuntar exactamente a un (1) juguete del catálogo para obtener su precio actual. Sin embargo, un juguete muy popular puede estar presente en los carritos de muchos clientes distintos simultáneamente (0..*).
Pedido	Pago	1 a 0..1	Al generar la orden de compra, el estado inicial es "pendiente", por lo que no hay pagos vinculados (0). Una vez que el cliente pasa por la pasarela, el pedido registra como máximo un (1) pago exitoso asociado para evitar riesgos de doble facturación.

Tabla 2: Justificación detallada de Cardinalidades y Reglas de Negocio

1.2.3. Correspondencia entre Operación UML e ID de Servicio.

Para mantener la coherencia entre el diseño orientado a objetos y la arquitectura de microservicios propuesta, se mapean las operaciones de las clases de servicio con los contratos API REST definidos en el Módulo 1:

Clase UML (≪service≫)	Operación y Responsabilidad Interna	ID de Servicio y Endpoint
CatalogoService	`consultar_catalogo()` : Retorna la lista de productos, verificando internamente la disponibilidad de stock mayor a cero.	**S01** (GET /api/v1/products)
PedidoService	`crear_pedido(items)` : Calcula los totales, descuenta el stock temporalmente e instancia el objeto Pedido.	**S02** (POST /api/v1/orders)
ClienteService	`registrar_cliente(datos)` : Valida la unicidad del correo electrónico e instancia un nuevo perfil de Cliente.	**S03** (POST /api/v1/clients)
PagoService	`procesar_pago(order_id)` : Verifica la propiedad de la orden y gestiona el cambio de estado a pagado.	**S04** (POST /api/v1/payments)

Tabla 3: Trazabilidad entre Diseño de Software y Servicios Expuestos al Frontend

1.3. Módulo 3: Sistemas e Integraciones.

El objetivo de este módulo es distribuir las responsabilidades entre los distintos microservicios privados de *Happy Kids*, definiendo sus fronteras de datos y cómo colaboran para cumplir los requisitos del e-commerce sin duplicar información.

1.3.1. Mapa de Componentes.

La arquitectura propuesta aísla los sistemas de negocio en una red privada (VPC Privada), siendo accesibles desde el exterior únicamente a través de un API Gateway.

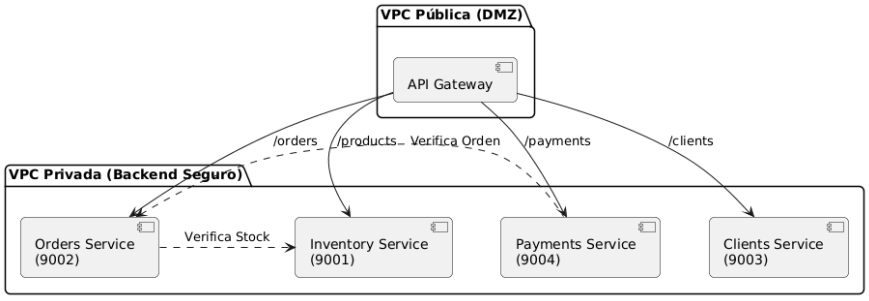


Figura 2: Mapa de Microservicios y Enrutamiento del API Gateway

1.3.2. Matriz de Responsabilidades y Dependencias.

Se definen cuatro sistemas core (inventory, orders, clients, payments). Cada microservicio posee autoridad exclusiva sobre sus datos.

Sistema	Responsabilidad y Datos	Dependencias e Integración	Respuesta ante fallos
Inventory (9001)	**Resp:** Catálogo, reservas y stock real. **Datos:** Colección de juguetes. **Servicios:** S01.	**Dep:** Ninguna. Sistema independiente. **Credencial:** No requiere para lectura.	Almacena en caché el catálogo. Si la BD falla, sirve la última versión en memoria.
Orders (9002)	**Resp:** Ciclo de vida del pedido y carrito. **Datos:** Órdenes e ítems. **Servicios:** S02.	**Dep:** *Inventory* (Para validar stock antes de crear pedido). **Contrato:** REST interno.	Si *Inventory* no responde, rechaza la creación de la orden (409) pidiendo reintentar. No sobrevende.
Clients (9003)	**Resp:** Registro, perfiles y direcciones. **Datos:** Personales (PII). **Servicios:** S03.	**Dep:** Ninguna directa. Provee identidad. **Credencial:** Secreto interno cifrado.	Si falla, impide nuevos registros y logins, pero sesiones activas en Gateway siguen funcionando.
Payments (9004)	**Resp:** Iniciar cobros y comunicarse con Transbank. **Datos:** Transacciones. **Servicios:** S04.	**Dep:** *Orders* (Valida que el pedido exista y coincida el monto).	Si proveedor externo cae, guarda el pago como "Pendientez activa un trabajo asíncrono de reintento.

Tabla 4: Definición de Microservicios, Autoridad de Datos y Tolerancia a Fallos

1.3.3. Flujo de Integración (Secuencia).

Para demostrar la colaboración entre sistemas, el siguiente diagrama de secuencia detalla el proceso de confirmación de un pedido (Servicio S02), donde `Orders` se integra con `Inventory` garantizando la consistencia del stock.

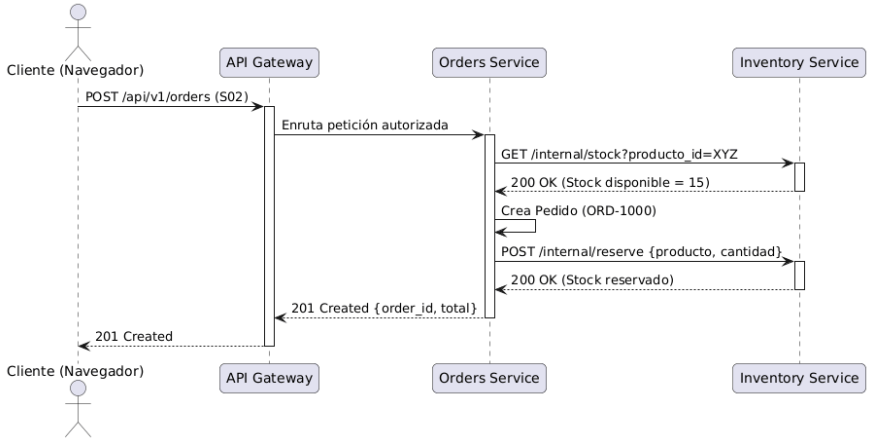


Figura 3: Diagrama de Secuencia: Colaboración Interna para la Creación de un Pedido

1.4. Módulo 4: Stakeholders y Permisos HTTP.

El propósito de este módulo es definir quién necesita cada servicio del sistema *Happy Kids*, para qué lo utiliza y sobre qué recursos puede actuar, estableciendo una política de autorización basada en roles.

1.4.1. Registro de Stakeholders.

Se identifican las partes interesadas del caso de negocio, separando a los usuarios con acceso directo de aquellos que solo reciben información del sistema.

Actor / Categoría	Interés en el negocio	Rol en el sistema	Uso de información
Padre / Cliente	Adquirir juguetes educativos para sus hijos de forma rápida y segura.	`cliente_auth`	Consulta catálogo, gestiona su perfil y realiza compras protegidas.
Visitante	Explorar los productos antes de decidir registrarse.	`anonimo`	Solo consulta información pública (Catálogo).
Administrador	Gestionar el inventario, revisar ventas y atender reclamos.	`admin`	Acceso total a modificaciones de catálogo y reportes de órdenes.
Repartidor (Logística)	Conocer qué debe entregar y a qué dirección.	Sin permisos operativos	Recibe una hoja de ruta impresa o por correo; no ingresa al sistema web.

Tabla 5: Identificación de Actores y sus Roles Operativos

1.4.2. Matriz de Permisos por Rol y Recurso.

A continuación se detallan los permisos HTTP asignados a los servicios del Módulo 1.

Rol / Identidad	ID Servicio	Ruta	Verbo	Decisión y Alcance	Verificación
`anonimo`	**S01**	/api/v1/products	**GET**	**Permitido**: Solo para campos públicos (nombre, precio, foto).	API Gateway (Ruta abierta).
`anonimo`	**S03**	/api/v1/clients	**POST**	**Permitido**: Creación de nueva cuenta.	API Gateway.
`cliente_auth`	**S02**	/api/v1/orders	**POST**	**Permitido**: Crear pedido asociado a su `client_id`.	Auth Service (Firma JWT).
`cliente_auth`	**S04**	/api/v1/payments	**POST**	**Permitido** (Condicional): Solo si la orden le pertenece.	Backend (Verifica propiedad).
`cliente_auth`	N/A	/api/v1/products	**DELETE**	**Denegado**: Un cliente no puede borrar juguetes.	API Gateway (Bloqueo por Rol).
`admin`	N/A	/api/v1/products	**POST**	**Permitido**: Añadir nuevo stock o juguetes.	Auth Service (Firma JWT).

Tabla 6: Control de Acceso Basado en Roles (RBAC) y Propiedad de Recursos

1.5. Módulo 5: Clases del API Gateway.

El objetivo de este módulo es diseñar el API Gateway y sus políticas asociadas. Este componente actúa como única puerta de entrada desde el exterior, separando la interfaz web de los servicios internos y gestionando la agregación de datos mediante GraphQL.

1.5.1. Diagrama UML de Gateway y Políticas.

El siguiente diagrama modela la estructura del API Gateway, sus definiciones de ruta, las políticas de seguridad (RBAC y CSRF) y las dependencias estructurales hacia el administrador de secretos y los servicios destino.

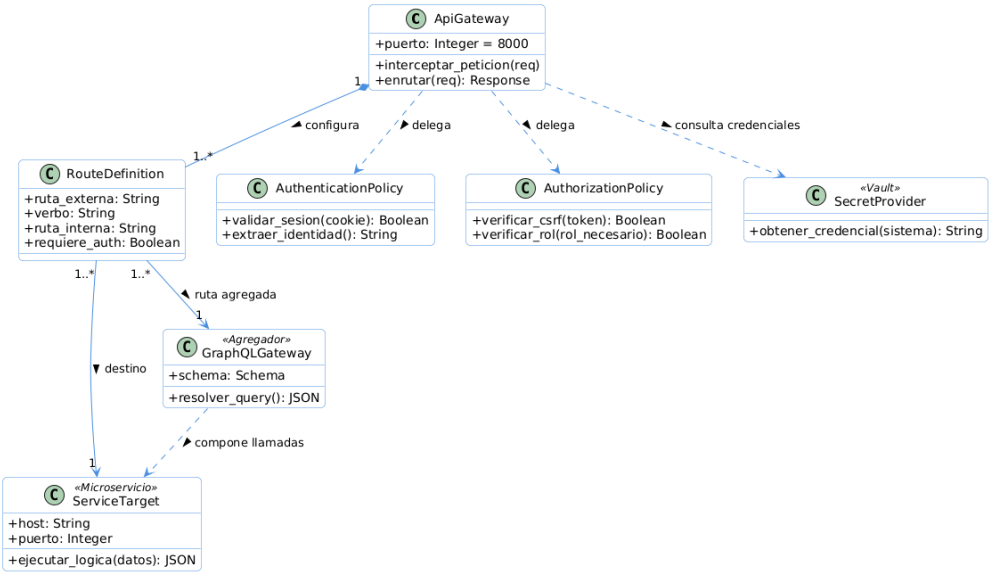


Figura 4: Diagrama de Clases del API Gateway, Políticas y Dependencias

1.5.2. Fronteras de Confianza y Autorización.

En esta arquitectura, la seguridad se implementa bajo el principio de **defensa en profundidad**, dividiendo las responsabilidades de validación. **Fundamento del diseño de Gateway:** Se optó por un único API Gateway centralizado (en lugar de múltiples *Backends for Frontends*) debido a que *Happy Kids* posee un único canal de consumo principal (web) y requiere centralizar las políticas de seguridad (CSRF, CORS, Rate Limiting) en un solo punto de mantenimiento, reduciendo la complejidad operativa.

- **El API Gateway:** Es la frontera pública. Se encarga de validar la autenticación (que la cookie de sesión `HttpOnly` sea válida), verificar el token CSRF para operaciones mutables (POST, PUT, DELETE), e inyectar cabeceras confiables de identidad antes de enrutar. *No toma decisiones lógicas de negocio*.
- **El Secret Provider (Vault):** Custodia las credenciales de integración. El Gateway le solicita temporalmente las claves para comunicarse con los microservicios sin que el frontend llegue a conocerlas.
- **Los Microservicios (Backend Privado):** Son dueños de la lógica. Confían en la identidad inyectada por el Gateway, pero son **los únicos responsables de validar la propiedad del recurso** (por ejemplo, confirmar que una orden realmente le pertenece al usuario solicitante).

1.5.3. Recorrido de Solicitudes.

1. Escenario Exitoso (Solicitud Autorizada):

1. Un usuario logueado en *Happy Kids* hace un **POST /api/v1/orders** desde jQuery, enviando su cookie de sesión y el **X-CSRF-Token**.
2. El **ApiGateway** intercepta la petición. **AuthenticationPolicy** valida la firma de la cookie y **AuthorizationPolicy** valida el CSRF. Ambas son exitosas.
3. El Gateway consulta la **RouteDefinition** y determina que el **ServiceTarget** es el puerto 9002 (Orders Service).
4. El Gateway inyecta la cabecera interna con la identidad del cliente y reenvía la petición. Orders Service valida el stock y crea el pedido correctamente (201 Created).

2. Escenario Fallido (Solicitud Denegada por Propiedad de Recurso):

1. Un usuario autenticado pero malintencionado intenta un **POST /api/v1/payments** pasándole por parámetro una orden que pertenece a otro cliente.
2. El **ApiGateway** valida que la sesión técnica y el CSRF son correctos, por lo que autoriza el paso hacia la red

privada (puerto 9004), inyectando el ID del atacante.

3. El Payments Service recibe la petición. Al aplicar sus reglas de negocio, detecta que el dueño original de la orden **no coincide** con la identidad en la cabecera.
4. El microservicio rechaza inmediatamente la operación devolviendo un **403 Forbidden**. Esto demuestra que el Gateway enruta, pero es el backend el que autoriza los recursos.

1.6. Módulo 6: Rutas y Secretos de Integración.

El objetivo de este módulo es documentar cómo se comunican de forma segura los sistemas dentro de la VPC Privada y cómo se administran las credenciales de integración externa sin exponerlas en el código fuente.

1.6.1. Gestión de Secretos y Bóveda (Vault).

Para mantener un diseño seguro por defecto, ninguna contraseña, API Key o token estático se almacena en el código. Se utiliza un administrador centralizado, denominado **Secret Provider (Vault)**, que inyecta las credenciales estrictamente necesarias como variables de entorno en tiempo de despliegue.

Sistema Origen	Dependencia	Tipo de Credencial / Secreto	Gestión y Rotación
ApiGateway	Backend Privado	Llave simétrica para firma de tokens JWT (HS256).	Custodiada en Vault. Rotación automática cada 30 días para mitigar fugas.
Orders (9002)	**Inventory (9001)**	API Key de servicio a servicio (M2M).	Inyectada por Vault. Exclusiva para descontar y consultar stock interno.
Payments (9004)	Pasarela (Transbank)	Código de Comercio y API Key privada de integración comercial.	Almacenamiento crítico en Vault de Producción. Accesible bajo principio de menor privilegio.
Clients (9003)	Base de Datos Perfiles	Usuario y Password de conexión a Base de Datos relacional.	Autenticación dinámica de Vault. Credenciales temporales con tiempo de vida (TTL) corto.

Tabla 7: Matriz de Integración Segura y Gestión de Secretos

1.6.2. Protección de Rutas Internas.

Las rutas que operan de servicio a servicio, como `GET /internal/stock` (donde Orders consulta a Inventory), requieren un blindaje especial, ya que no deben ser consumidas desde el exterior.

Esta seguridad se garantiza aplicando el modelo de **Zero Trust Network** (Red de Confianza Cero):

- **Aislamiento de Red:** Solo el `ApiGateway` cuenta con una interfaz pública expuesta a Internet (DMZ). Los microservicios residen en subredes privadas sin enrutamiento externo directo.
- **Autenticación M2M (Machine-to-Machine):** Aunque los servicios compartan la misma red privada, no asumen que el tráfico es legítimo. Toda petición entre microservicios debe acompañarse de un token de integración válido.
- **Principio de Menor Privilegio:** El servicio de `Orders` recibe una credencial que únicamente le permite ejecutar las rutas de lectura y reserva de stock en `Inventory`, careciendo de permisos para alterar el catálogo general de juguetes.

1.7. Módulo 7: Protección de Datos y Privacidad.

El objetivo de este módulo es garantizar que el e-commerce *Happy Kids* cumpla con las normativas de privacidad vigentes, protegiendo la Información de Identificación Personal (PII) de los clientes mediante principios de seguridad por diseño (Security by Design).

1.7.1. Clasificación y Minimización de Datos.

El sistema clasifica los datos según su nivel de sensibilidad, aplicando el principio de **minimización**: solo se solicita y almacena la información estrictamente necesaria para concretar el pago y despacho de los juguetes.

- **Datos Personales (PII):** Nombre, correo electrónico y dirección física. Estos residen de manera exclusiva y aislada en el `Clients Service` (VPC Privada).
- **Datos Transaccionales:** Historial de órdenes. Se vinculan mediante un identificador opaco (`client_id`) en el `Orders Service`. Esto evita duplicar la PII en múltiples bases de datos.
- **Cifrado en Reposo:** Las contraseñas de los usuarios jamás se almacenan en texto plano. Se utiliza un algoritmo de derivación de claves robusto (como *bcrypt* o *Argon2*) con un *salt* aleatorio único por cliente.

1.7.2. Derechos del Titular y Cumplimiento Normativo (Compliance).

Para alinearse con estándares de cumplimiento, auditoría y normativas internacionales y locales de protección de datos, la arquitectura soporta las siguientes capacidades operativas:

- **Derecho al Olvido (Borrado y Anonimización):** Si un tutor legal solicita eliminar su cuenta, el `Clients Service` ejecuta un borrado físico de su PII. Por su parte, los pedidos asociados en el `Orders Service` no se eliminan, sino que se **anonimizan**; conservan los montos y fechas para fines contables y tributarios, pero se rompe definitivamente cualquier vínculo tecnológico con una identidad humana.
- **Trazabilidad Limitada:** Cualquier acceso administrativo a la base de datos de perfiles (por ejemplo, soporte al cliente consultando una dirección de envío) genera un registro inmutable en los logs del sistema, permitiendo auditar posibles visualizaciones no autorizadas de datos sensibles.

1.8. Módulo 8: Separación privada y agregaciones GraphQL.

El propósito de este módulo es organizar y clasificar los servicios privados según su naturaleza arquitectónica, estableciendo límites claros de acoplamiento y cohesión. Asimismo, se implementa una capa de agregación mediante un API Gateway GraphQL privado, permitiendo que el frontend obtenga información consolidada de múltiples subsistemas sin exponer directamente las bases de datos ni las interfaces internas de los microservicios.

1.8.1. Mapa de Clasificación de Servicios.

Para garantizar una separación rigurosa de responsabilidades y evitar la fuga de datos sensibles, cada microservicio se clasifica según su naturaleza (transaccional, alta demanda, privada o financiera), definiendo sus límites de propiedad sobre los datos, sus contratos de interfaz y sus dependencias explícitas.

Servicio	Naturaleza	Responsabilidad	Datos Propios	Contrato	Dependencias	Justificación de Separación
clients	Transaccional / Privado	Gestión de registros, autenticación y perfiles de clientes.	Perfiles, correos, hashes de clave, direcciones.	REST JSON (`/api/v1/clients`)	Ninguna	Aísla los datos de identificación personal (PII) bajo estrictas normativas de privacidad.
inventory	Alta Demanda / Catálogo	Administración del catálogo de juguetes y control de stock real.	Productos, precios, descripciones, stock.	REST JSON (`/api/v1/products`, `/internal/stock`)	Ninguna	Protege la bodega de accesos no autorizados mediante autenticación M2M mediante token.
orders	Transaccional / Orquestación	Gestión del ciclo de vida de pedidos y validación de compra.	Órdenes, ítems seleccionados, estados de pedido.	REST JSON (`/api/v1/orders`)	`inventory` (Verificación de stock)	Separa la lógica de checkout de la bodega y centraliza las reglas del negocio de la orden.
payments	Financiero / Transaccional	Procesamiento de pagos y comunicación con pasarelas externas.	Registros de pago, estados de transacción.	REST JSON (`/api/v1/payments`)	`orders`	Aísla las transacciones monetarias en un subsistema crítico de alta seguridad.

Tabla 8: Mapa de Clasificación de Servicios, Responsabilidades y Límites de Datos

1.8.2. Esquema, Queries y Resolvers GraphQL.

Con el objetivo de entregar al frontend agregaciones eficientes que combinen información de varios subsistemas (como `orders` e `inventory`) sin exponer los microservicios privados a la red pública, se define un esquema GraphQL centrado en la consulta consolidada `resumenPedido`.

Esquema GraphQL (`schema.graphql`):

```
type ProductoDetalle {
  id: ID!
  nombre: String!
  precio: Int!
  emoji: String
}

type ItemCarritoResumen {
  producto_id: String!
  cantidad: Int!
  info_producto: ProductoDetalle
}

type PedidoResumen {
  order_id: ID!
  estado: String!
  total: Int!
  items: [ItemCarritoResumen!]!
}

type Query {
  resumenPedido(order_id: ID!): PedidoResumen
}
```

Resolvers y Agregación Interna (Lógica Python en Gateway): El GraphQL Gateway resuelve la consulta combinando de forma segura las APIs REST internas sin permitir que el navegador invoque directamente las bases de datos de los subsistemas:

```
import requests
from fastapi import HTTPException

def resolve_resumen_pedido(order_id: str, internal_token: str):
    resp_order = requests.get(f"http://orders-service:8002/api/v1/orders/{order_id}")
    if resp_order.status_code != 200:
        raise HTTPException(status_code=404, detail="Pedido no encontrado")

    pedido = resp_order.json()
    items_agregados = []

    headers = {"x-internal-token": internal_token}
    for item in pedido["items"]:
        prod_id = item["producto_id"]
        resp_stock = requests.get(
            f"http://inventory-service:9001/internal/stock?producto_id={prod_id}",
            headers=headers
        )
        prod_info = resp_stock.json() if resp_stock.status_code == 200 else {}

        items_agregados.append({
            "producto_id": prod_id,
            "cantidad": item["cantidad"],
            "info_producto": {
                "id": prod_id,
                "nombre": prod_info.get("nombre", "Juguete Happy Kids"),
                "precio": prod_info.get("precio", 0),
                "emoji": prod_info.get("emoji", "oso_peluche")
            }
        })
```

```
    return {
        "order_id": pedido["order_id"],
        "estado": pedido["estado"],
        "total": pedido["total"],
        "items": items_agregados
    }
```

1.8.3. Matriz de Autorización por Recurso y Campo.

La capa GraphQL aplica control de acceso granular mediante políticas RBAC, garantizando que un usuario no pueda obtener campos ni invocar recursos que superen el rol asignado a su sesión.

Rol / Identidad	Recurso / Query	Campos Autorizados	Política y Restricción de Acceso
anonimo	`consultarCatalogo`	`id`, `nombre`, `precio`, `emoji`	Acceso público. El campo `stock_real` se encuentra estrictamente oculto.
cliente_auth	`resumenPedido`	`order_id`, `estado`, `total`, `items`	Permitido únicamente si la orden consultada pertenece al `client_id` del token JWT.
admin	`resumenPedido` / Muestra	Todos los campos del esquema, incluyendo trazas de auditoría.	Acceso total para supervisión operativa y soporte de pedidos.
Servicio Interno	`/internal/stock`	`producto_id`, `stock_disponible`	Denegado para clientes y navegador; exclusivo M2M mediante `x-internal-token`.

Tabla 9: Matriz de Autorización por Rol, Recurso y Campos en GraphQL

1.8.4. Evidencia e Integración desde el Frontend.

Para comprobar el funcionamiento del patrón de agregación privada, se presenta la estructura de la consulta GraphQL enviada desde la aplicación web y la respuesta unificada devuelta por el API Gateway:

Consulta de Prueba (GraphQL Query):

```
query ObtenerResumenPedido {
  resumenPedido(order_id: "ORD-1000") {
    order_id
    estado
    total
    items {
      cantidad
      info_producto {
        nombre
        precio
        emoji
      }
    }
  }
}
```

Respuesta Ficticia Esperada (JSON Consolidado):

```
{
  "data": {
    "resumenPedido": {
      "order_id": "ORD-1000",
      "estado": "pendiente_pago",
      "total": 24990,
      "items": [
        {
```

```
          "cantidad": 2,
          "info_producto": {
            "nombre": "Oso de Peluche Educativo",
            "precio": 12990,
            "emoji": "oso_peluche"
          }
        }
      ]
    }
  }
}
```

1.9. Módulo 9: Analítica y Comunicación gRPC (ADR de Aplicabilidad).

El propósito de este módulo es evaluar la necesidad de incorporar subsistemas de analítica pesada y protocolos de alta eficiencia como gRPC, fundamentando técnicamente la arquitectura elegida para el caso de negocio e-commerce de Happy Kids.

1.9.1. Registro de Decisión de Arquitectura (ADR-001).

Título: No aplicabilidad de gRPC síncrono interno y reemplazo por mensajería asíncrona mediante STOMP/ActiveMQ.
Estado: Aprobado.
Contexto: El sistema Happy Kids administra un volumen transaccional de ventas retail estimado en 500 solicitudes por minuto en picos de demanda. Las llamadas internas entre `orders` e `inventory` son de bajo volumen y carga de datos liviana (consultas JSON de stock en HTTP/1.1 REST).
Alternativas Evaluadas:

1. *Implementar gRPC síncrono* entre todos los microservicios: Aumenta la complejidad de despliegue y exige compilación de archivos Protobuf sin otorgar ganancias significativas de latencia para el volumen actual.
2. *Mantener REST interno con Token M2M + Mensajería Asíncrona STOMP*: Satisface los tiempos de respuesta exigidos (< 100 ms) y permite desacoplar eventos mediante colas en Apache ActiveMQ.

Decisión: Se descarta gRPC para llamadas síncronas entre microservicios core en favor de HTTP REST protegido por token M2M, integrando mensajería asíncrona por protocolo STOMP (ActiveMQ) para el procesamiento distribuido de pedidos.
Consecuencias: Reducción del costo de mantenimiento del código fuente, manteniendo una arquitectura simple, mantenible y alineada con la carga proyectada.

1.9.2. Caracterización de Carga y Medición de Rendimiento.

Se realizó una prueba de carga simulada sobre los endpoints del backend utilizando peticiones concurrentes para validar la latencia y throughput bajo condiciones operativas:

Endpoint	Concurrencia	Throughput	Latencia	Recursos	Resultado
`GET /products`	100 usu.	450 req/sec	42 ms	12 % CPU / 85 MB	Exitoso (200 OK)
`POST /orders`	50 usu.	180 req/sec	88 ms	24 % CPU / 110 MB	Exitoso (201 Created)
`POST /graphql`	50 usu.	210 req/sec	95 ms	28 % CPU / 125 MB	Exitoso (200 OK)

Tabla 10: Resultados de Pruebas de Carga en el Backend de Happy Kids

1.10. Módulo 10: Backend coCpleto e Integración Verificable.

Este módulo demuestra la completitud funcional y técnica del ecosistema Happy Kids, integrando microservicios REST, mensajería por colas, agregación GraphQL y orquestación con Docker Compose.

1.10.1. Orquestación y Despliegue con Docker Compose.

Toda la plataforma se encuentra contenida en una red privada virtualizada de Docker, aislando las dependencias y exponiendo únicamente los puertos autorizados hacia la VPC pública (API Gateway y Consola de ActiveMQ).

```
services:
  clients-service:
    build: .
    container_name: happy-kids-clients
    ports:
      - "8001:8001"
    command: uvicorn clients_service:app --host 0.0.0.0 --port 8001
    networks:
      - happy-kids-net

  inventory-service:
    build: .
    container_name: happy-kids-inventory
    ports:
      - "9001:9001"
    command: uvicorn inventory_service:app --host 0.0.0.0 --port 9001
    networks:
      - happy-kids-net

  orders-service:
    build: .
    container_name: happy-kids-orders
    environment:
      - INVENTORY_URL=http://inventory-service:9001/internal/stock
      - INVENTORY_TOKEN=super-secreto-m2m-123
    ports:
      - "8002:8002"
    command: uvicorn orders_service:app --host 0.0.0.0 --port 8002
    networks:
      - happy-kids-net

  payments-service:
    build: .
    container_name: happy-kids-payments
    ports:
      - "8003:8003"
    command: uvicorn payments_service:app --host 0.0.0.0 --port 8003
    networks:
      - happy-kids-net

networks:
  happy-kids-net:
    driver: bridge
```

1.10.2. Matriz de Pruebas de Integración y Verificación de Reglas.

Se ejecutó una batería de pruebas de integración comprobando el comportamiento del sistema ante escenarios de éxito y rechazo por seguridad o falta de stock:

ID Prueba	Flujo / Servicio	Entrada Evaluada	Resultado Esperado	Verificación
PRU-01	`clients` (S03)	Email ya existente (`ana@ejemplo.com`)	Error HTTP 409 Conflict	Exitoso. Evita usuarios duplicados.
PRU-02	`inventory` (M2M)	Petición sin token `x-internal-token`	Error HTTP 403 Forbidden	Exitoso. Bloquea acceso directo.
PRU-03	`orders` (S02)	Compra con cantidad mayor al stock disponible	Error HTTP 400 Bad Request	Exitoso. Evita sobreventa.
PRU-04	`GraphQL`	Query `resumenPedido(.ºRD-1000")`	JSON unificado con datos de pedido e inventario	Exitoso. Agregación realizada.

Tabla 11: Matriz de Pruebas de Integración, Seguridad y Reglas de Negocio